

Week 14: Cryptography Applications (P2)

PhD. Ngoc-Tu Nguyen

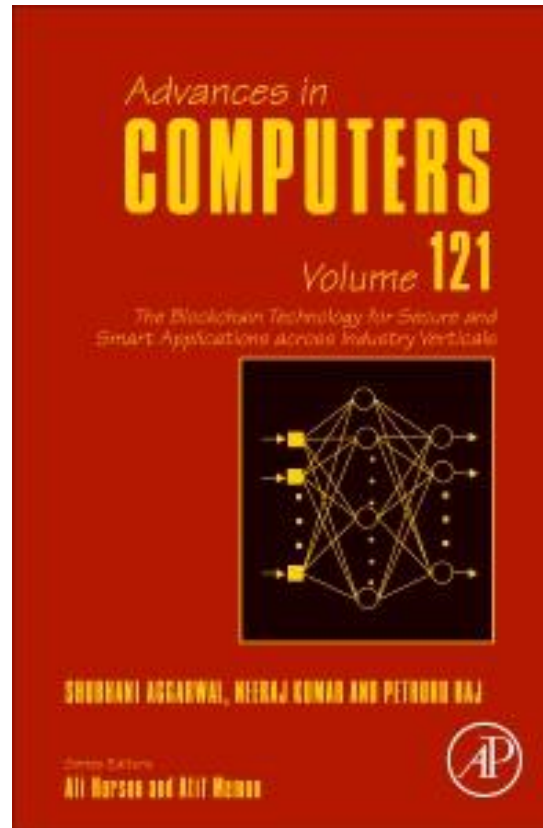
tunn@uit.edu.vn

Outline

- Network secure protocols
 - Authentication;
 - Key agreement;
 - IPSec; SSL/TLS; SSH; Kerberos
- Blockchain network security
 - Motivation
 - Database structure;
 - Secure Transactions and consensus mechanism;
 - Network architecture;
 - Applications

Textbooks and References

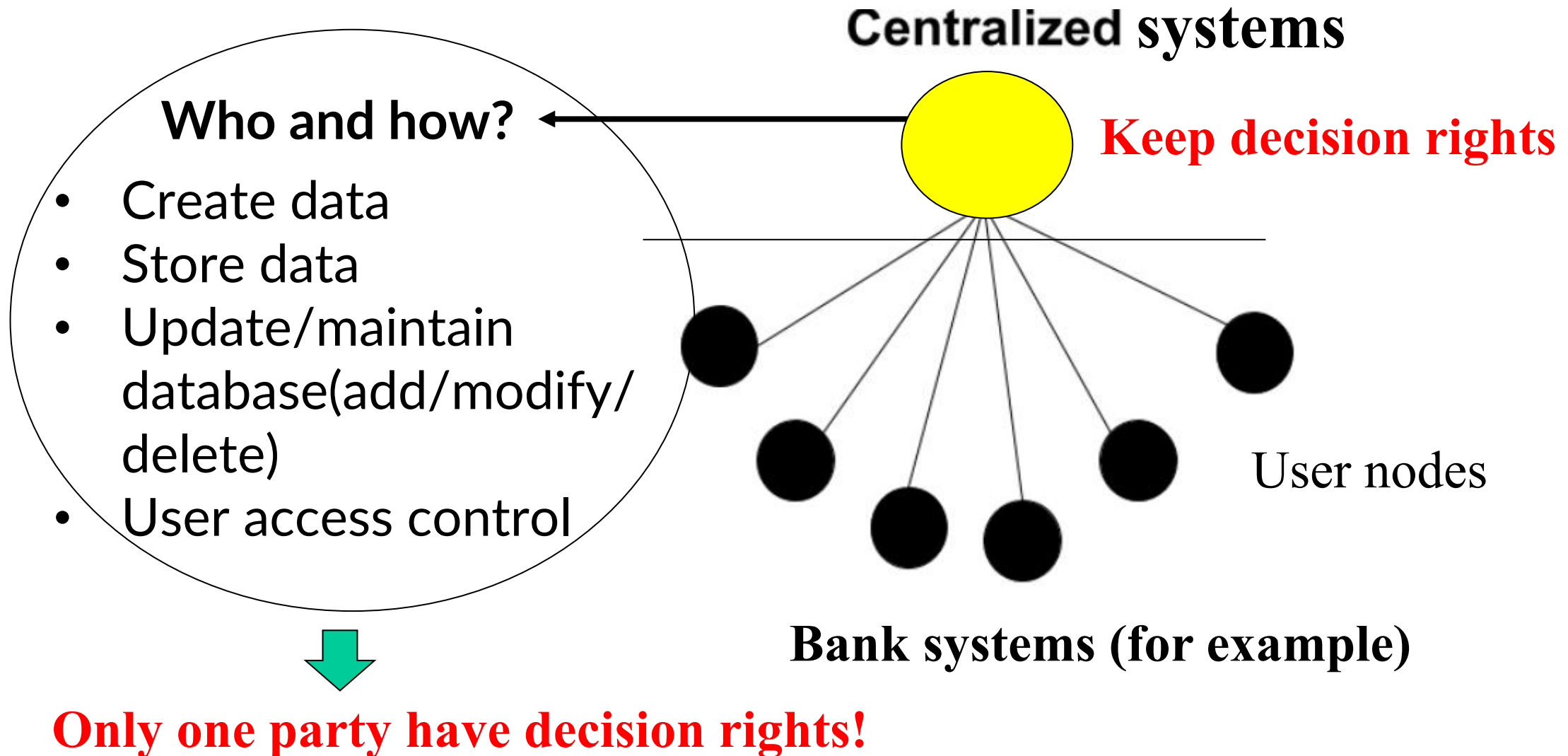
- Referent book



Kumar, N., Aggarwal, S., & Raj, P. (2021). The blockchain technology for secure and smart applications across industry verticals. Academic Press.

Outline (P2)

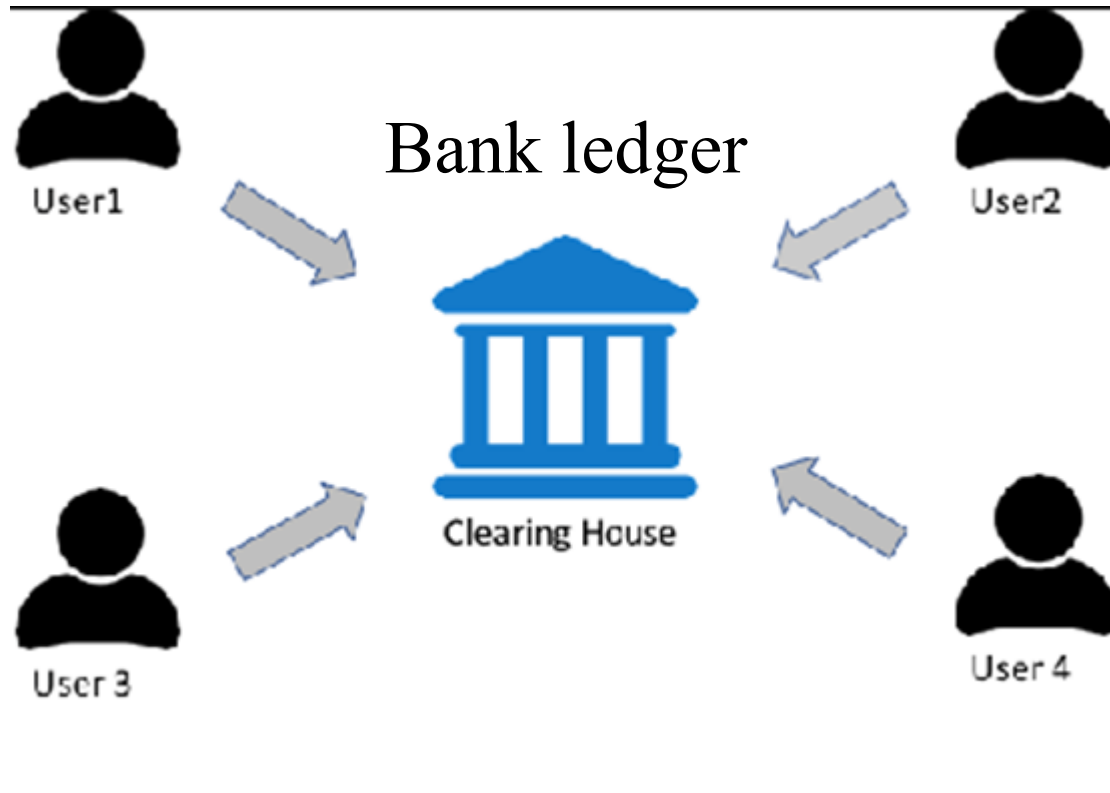
- **Blockchain network and applications**
 - Motivations
 - Hash-based secure storage (immutable database)
 - Signature-based authentication and verification (users/events):
 - Consensus mechanism and coin mining (majority-rule security)
 - Blockchain architecture: a bitcoin case study;
 - Next generation: Transaction protocol (smart contract) and distributed applications
 - Implementation and application sectors



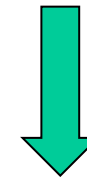
Motivation

Centralized authorities

User's records: transaction history,
account balance,...



**The bank controls
everything!**



**single point of
risk**

(dishonest, compromised)

Motivations

Centralized authorities

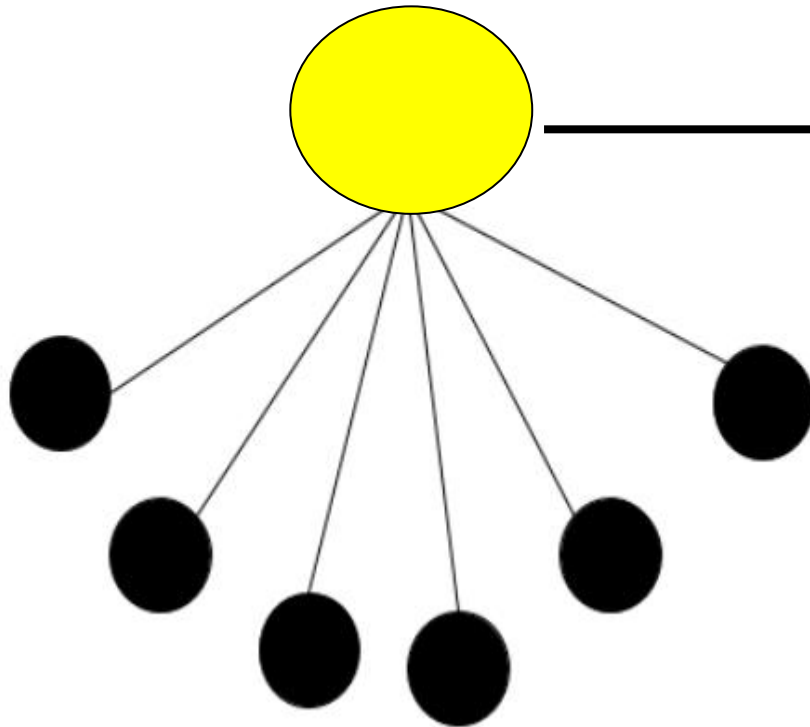
Only one party have decision rights!



➤ We don't know what they do but have to trust!

➤ Single point of risk (dishonest, compromised)

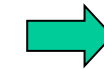
Centralized systems



- Create data
- Store data
- Update/maintain database(add,modify,delete)
- User access control



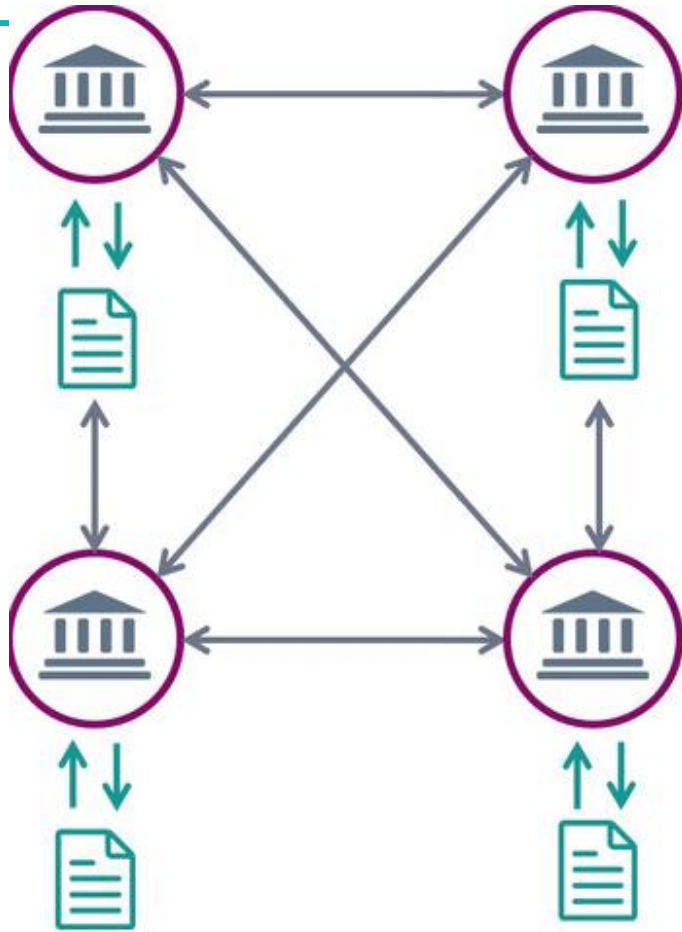
Trust or distrust
(spoofing)?



Secure?
(integrity, privacy
transparency, ...)

Motivations

Remove the single point of control



Ex. User's record: transaction history, account balance,...

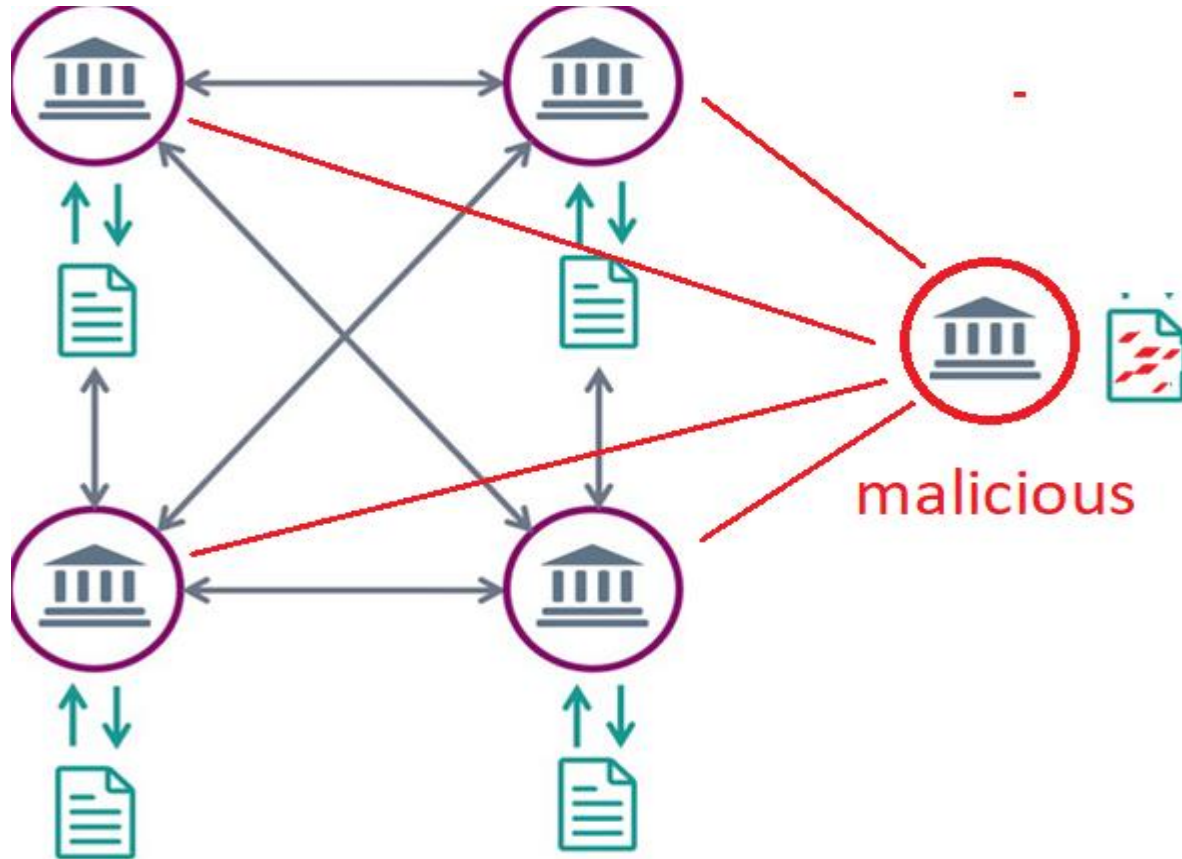
Distributed rights systems

- **Decision rights:** multiple nodes cooperate to make decisions for new events (~ votes in parliament)
- **Secure verification and storage:** multiple locations (all full nodes)

How to **verify data**, securely synchronize or record new events with error-tolerant (appear fake nodes)

- **Verifying data/transactions (security features)?**
- **Securely synchronize data (consensus mechanism)**
- ✓ **Create, verify, synchronize new events?**

Fault-tolerant mechanism



- **How to ?**
 - ✓ Verifying new events/transations (data authentication, other security features)
 - ✓ Securely storage, verify, synchronize new events (all nodes): immutability, transability,

Byzantine Fault Tolerance

- "Crash Fault Tolerance" (CFT) aims to guarantee the functioning of a distributed system in the presence of machines crashing or disconnecting and reconnecting at random
- "Byzantine Fault Tolerance" (BFT) aims to guarantee the ability of a group of nodes to achieve consensus in the presence of malicious actors who are trying to subvert the consensus process in their favour, or prevent consensus from being achieved.
- BFT is a much harder goal than CFT, and hasn't been solved mathematically at scale; BFT generally makes use of cryptographic signatures, etc.
- Approximations of BFT exist, but they all come with significant constraints and limitations attached

Outline

- Network secure protocols
 - Authentication;
 - Key agreement;
 - IPSec; SSL/TLS; SSH; Kerberos
- Blockchain network security
 - Motivation
 - Database structure;
 - Secure Transactions and consensus mechanism;
 - Network architecture;
 - Applications

Immutable database (Integrity)

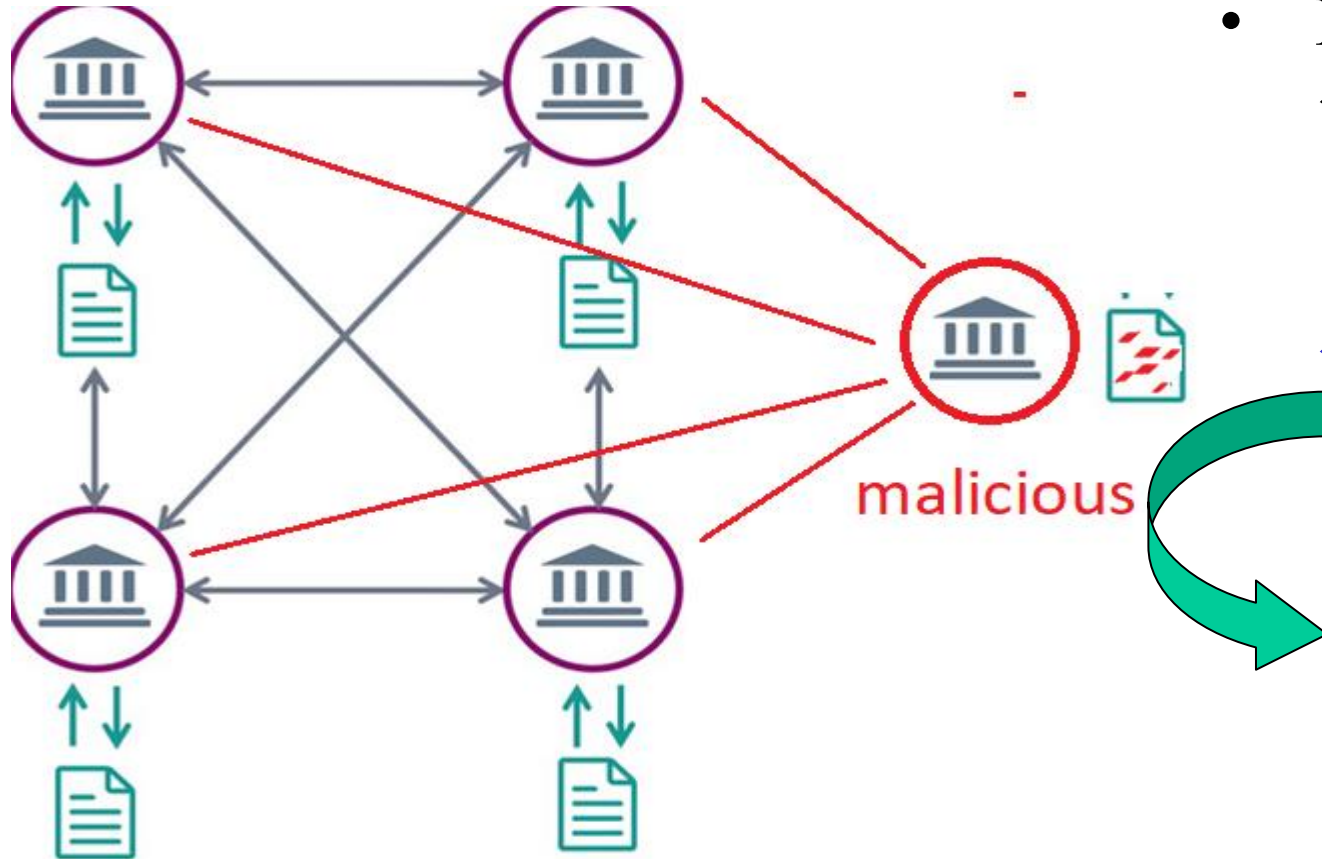
User's record:

- Owner
- **account balance,**
- **transaction records,**  need authenticate and secure storage (integrity, immutability)
- ...

✓ Common methods

- **Signature-based:** authenticate the related parties, and verify the integrity (original);
- **Hash-based:** Immutable storage
 - Hash trees
 - Hash chains

Immutable database (Integrity)



- **How to ?**

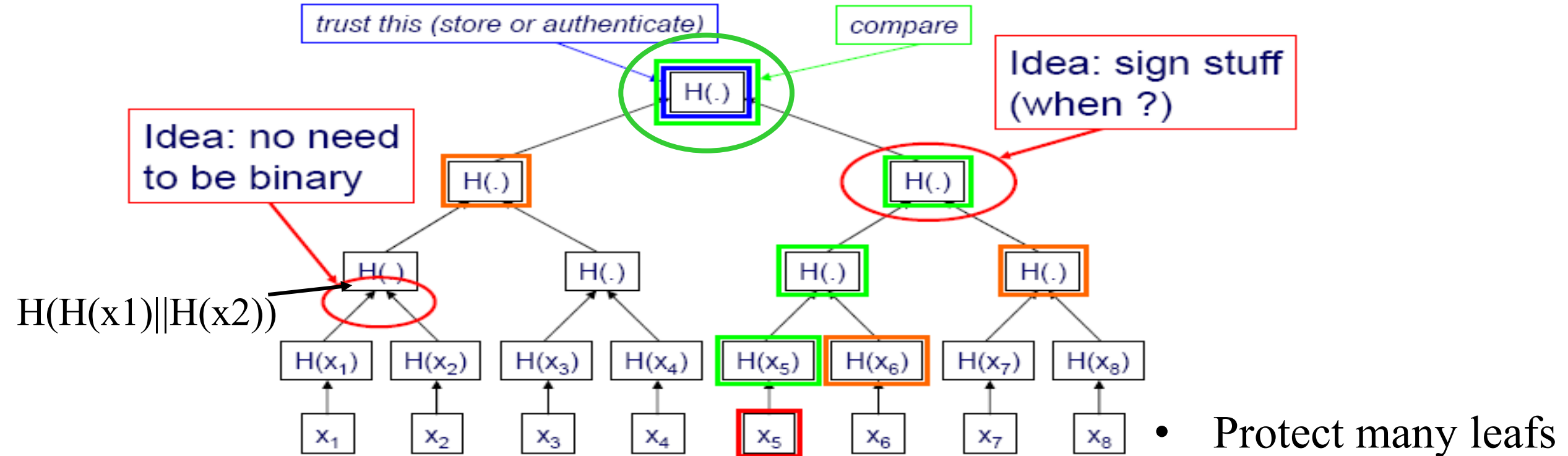
- ✓ Verifying new events/transations (data authentication, other security features)
- ✓ **Securely storage**, verify, synchronize new events (all nodes): immutability, transability,

Hashchain + Merkle hash tree
= blockchain

Immutable database (Integrity)

■ Merkle hash tree

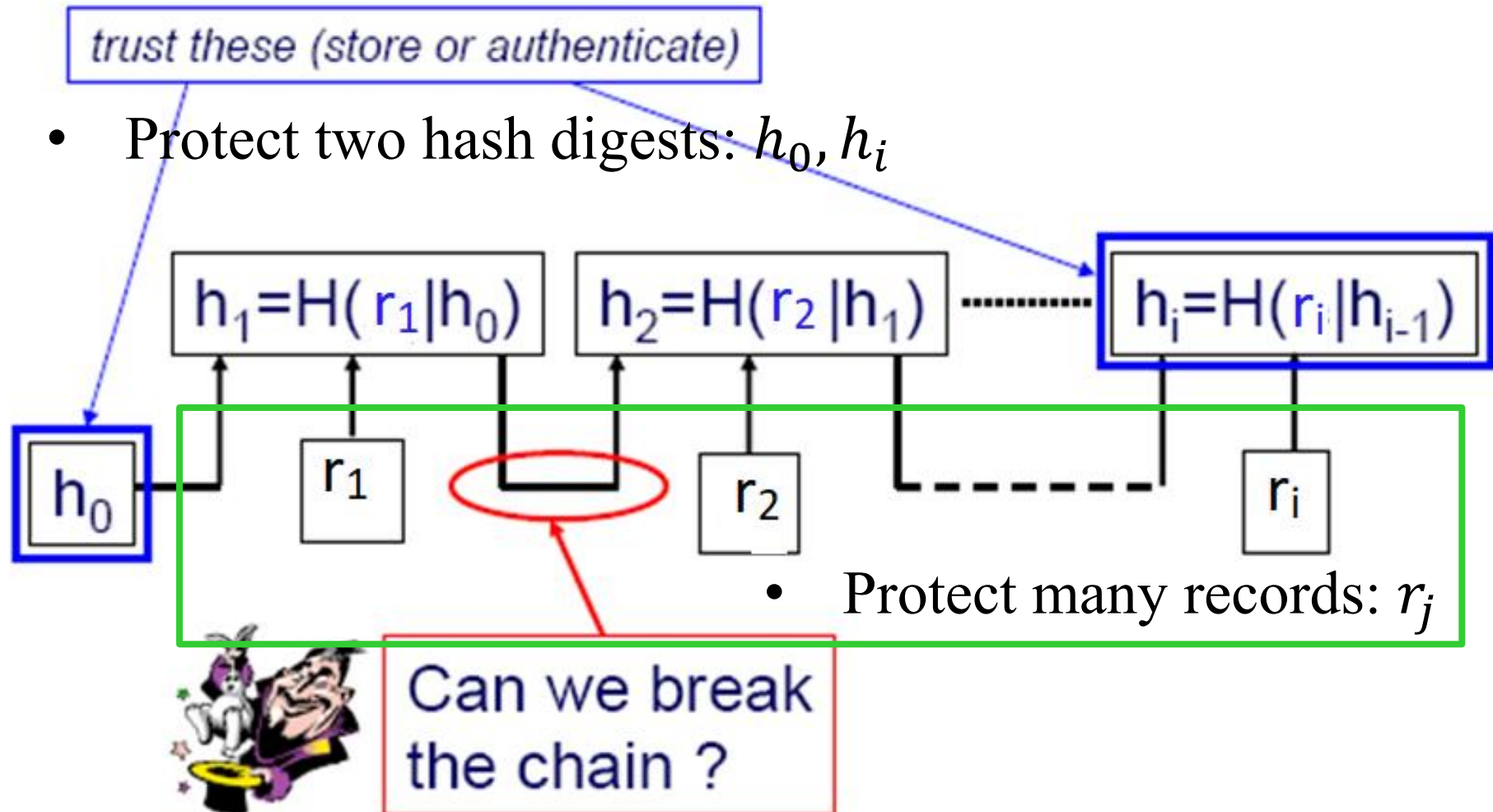
- → Protect one root



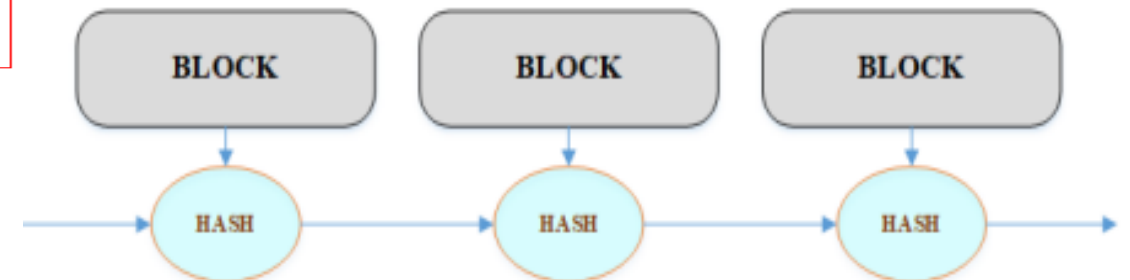
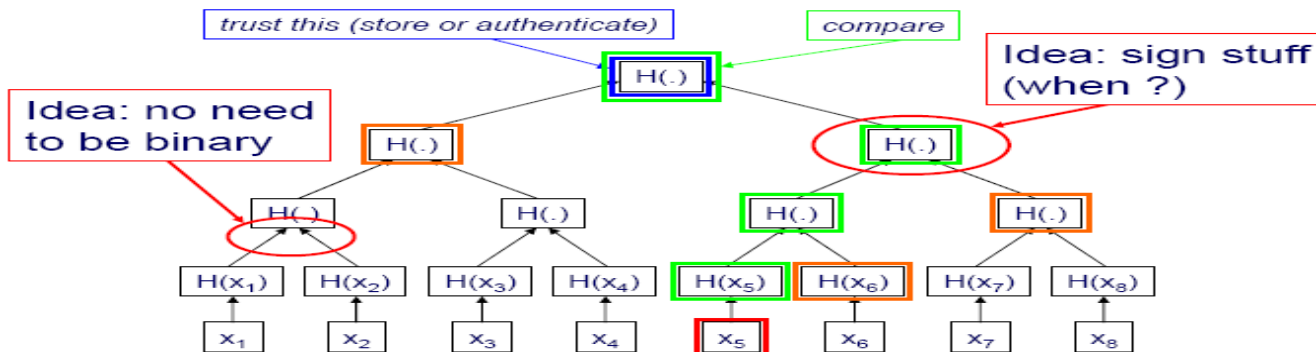
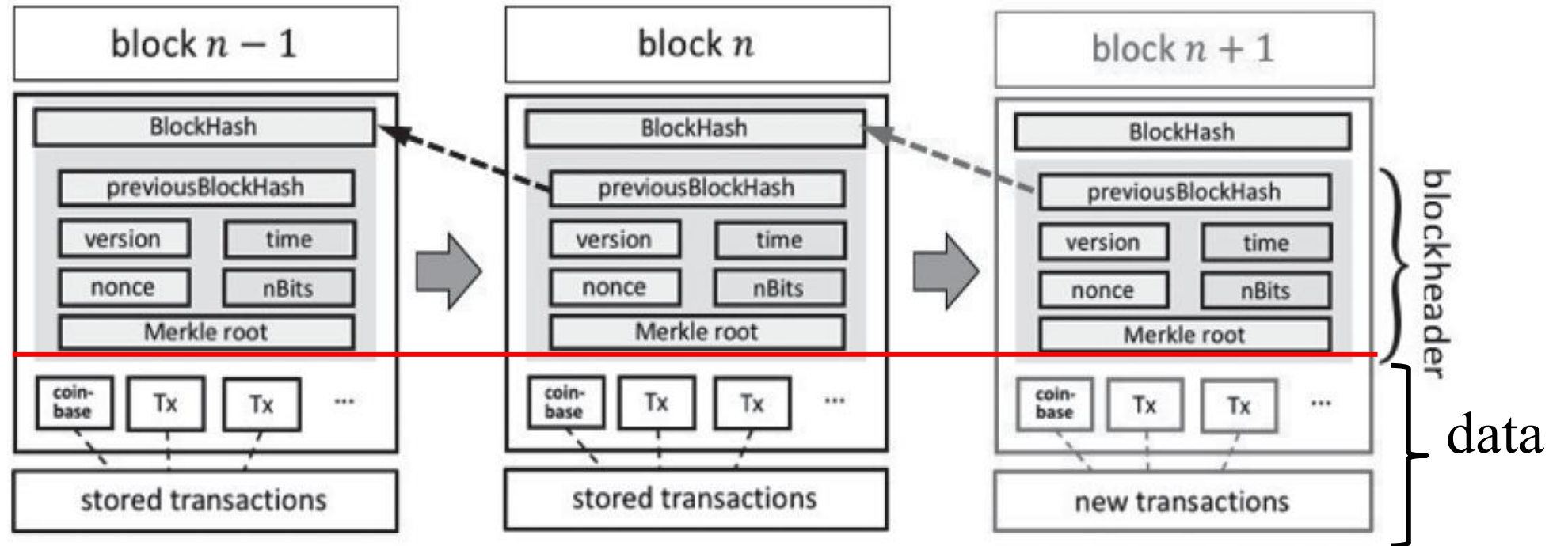
- “||” denote the string concatenation;
- Can be stored with tree like structure : index, xml

Immutable database (Integrity)

■ Hash chains

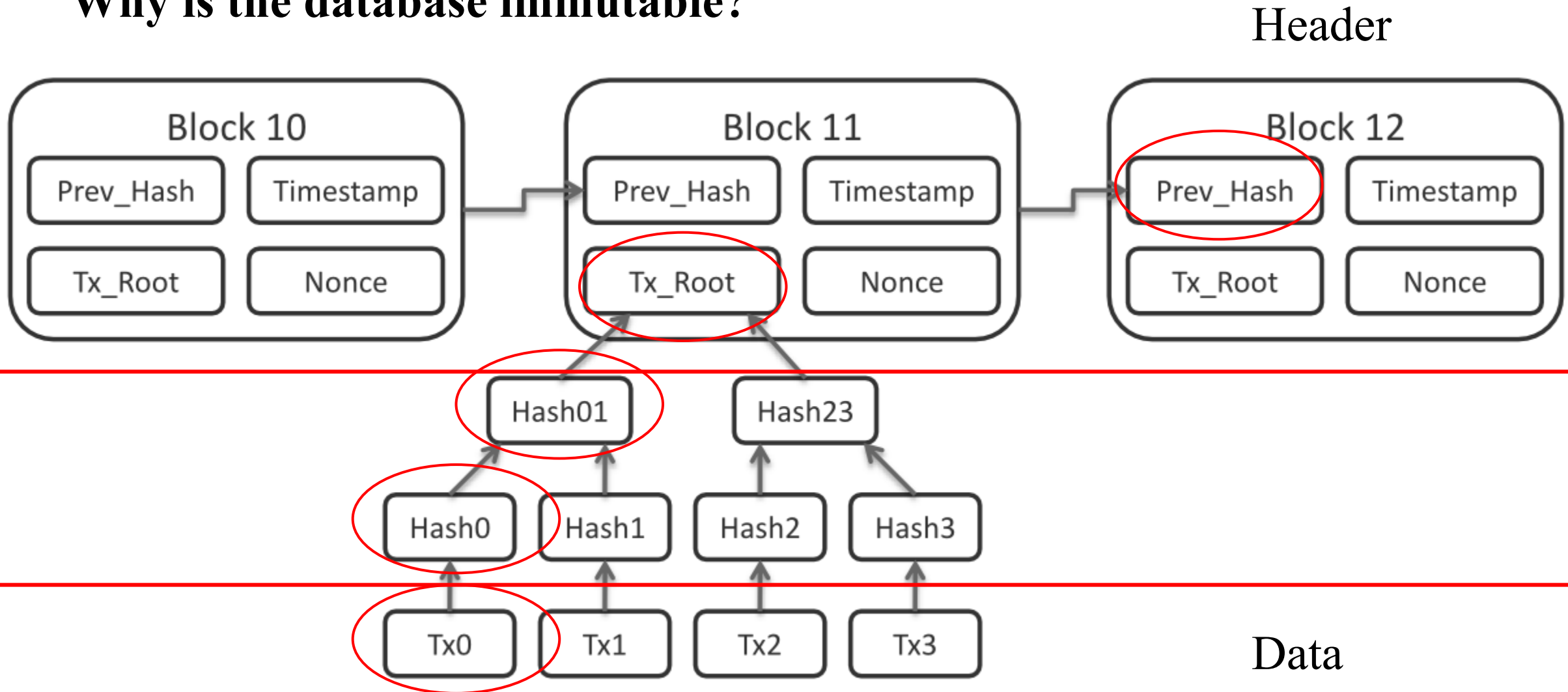


Bitcoin database: a case study



Bitcoin database: a case study

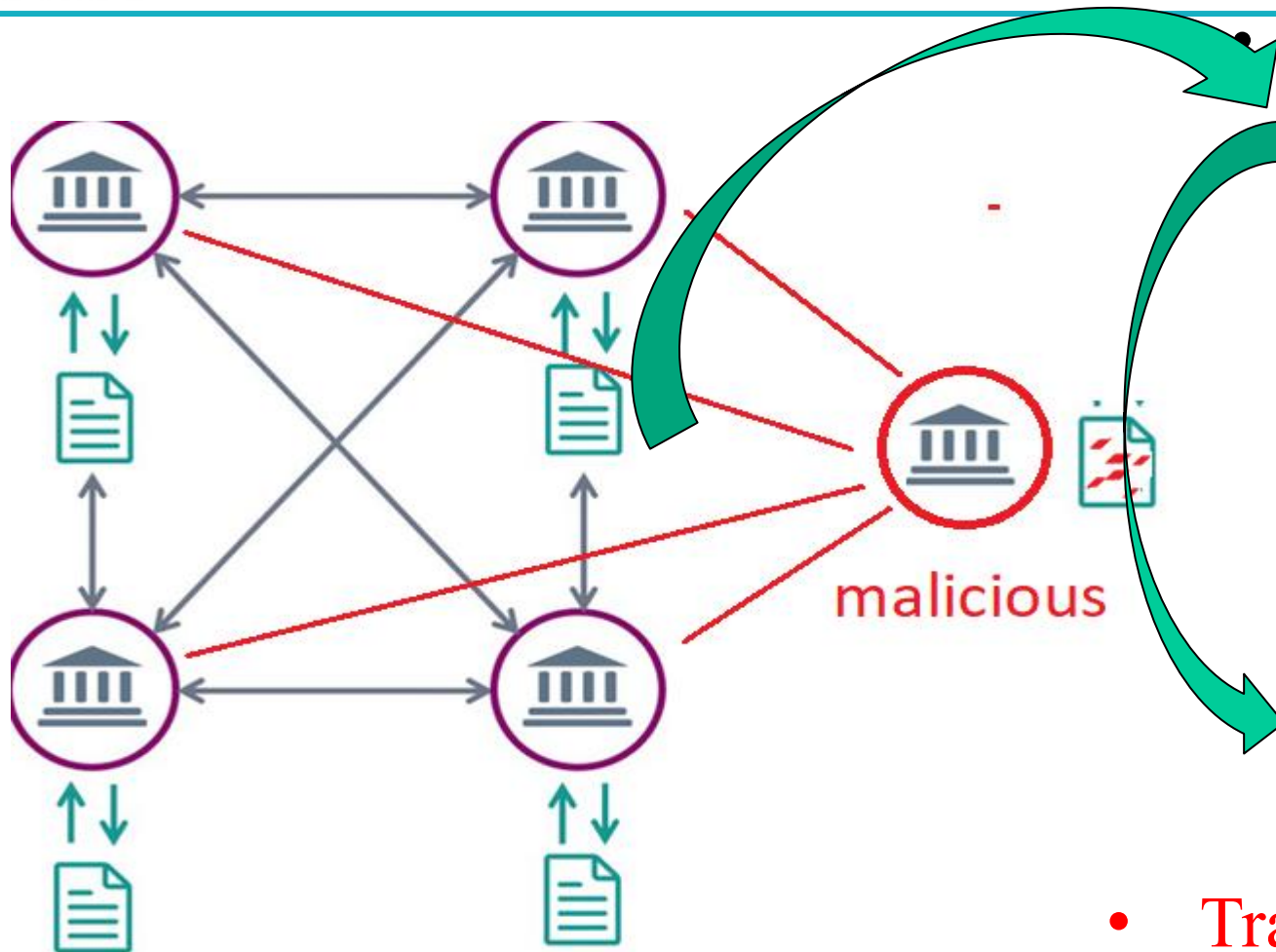
Why is the database immutable?



Outline

- Network secure protocols
 - Authentication;
 - Key agreement;
 - IPSec; SSL/TLS; SSH; Kerberos
- Blockchain network security
 - Motivation
 - Database structure;
 - **Secure Transactions and consensus mechanism;**
 - Network architecture;
 - Applications

Verifying transaction data



How to ?

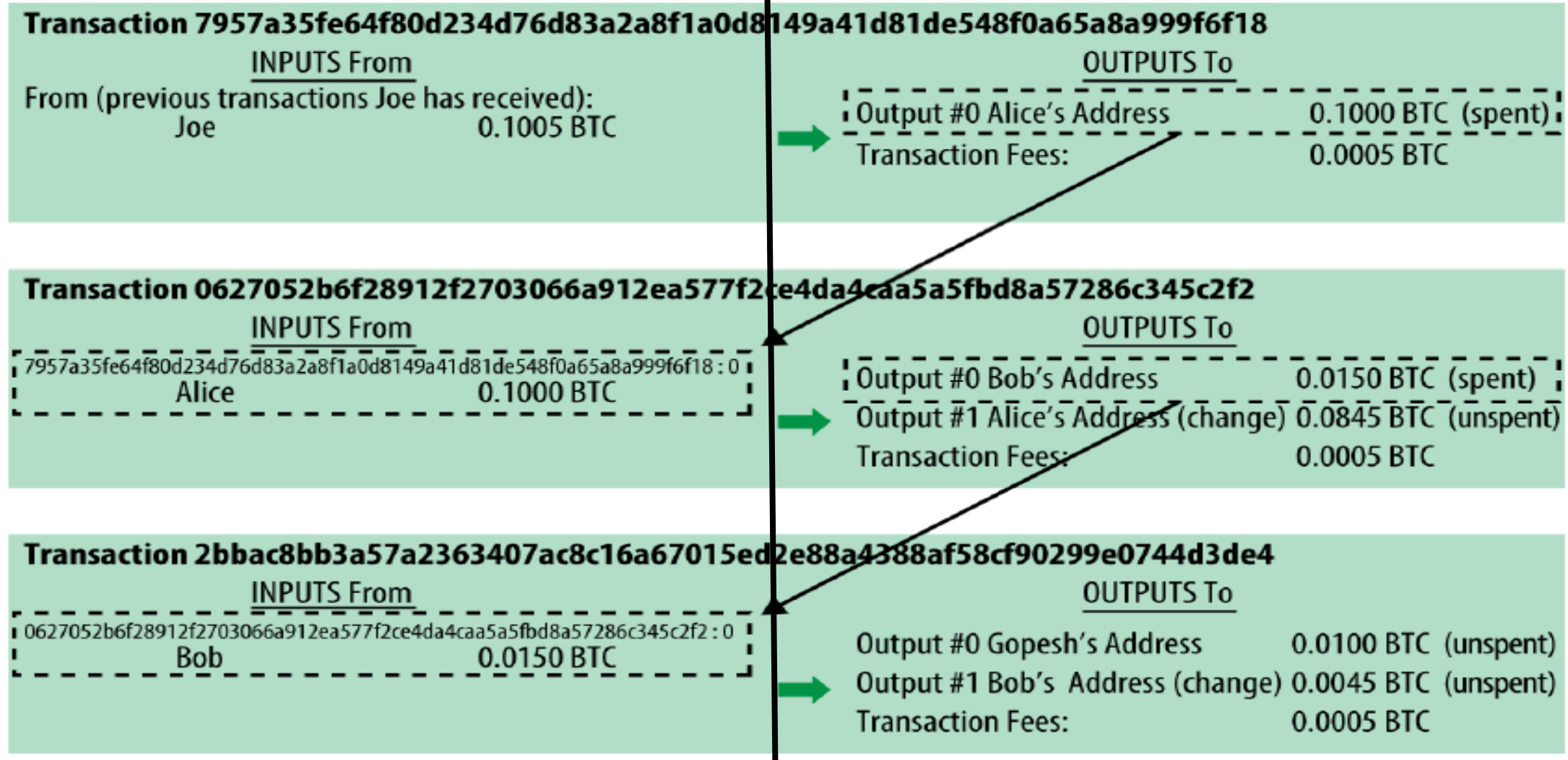
- ✓ Verifying new events/transactions (data authentication, other security features)
- ✓ Securely storage, verify, synchronize new events (all nodes): immutability, transability,

Signature-based: authenticate the related parties and data (sources, content, integrity);

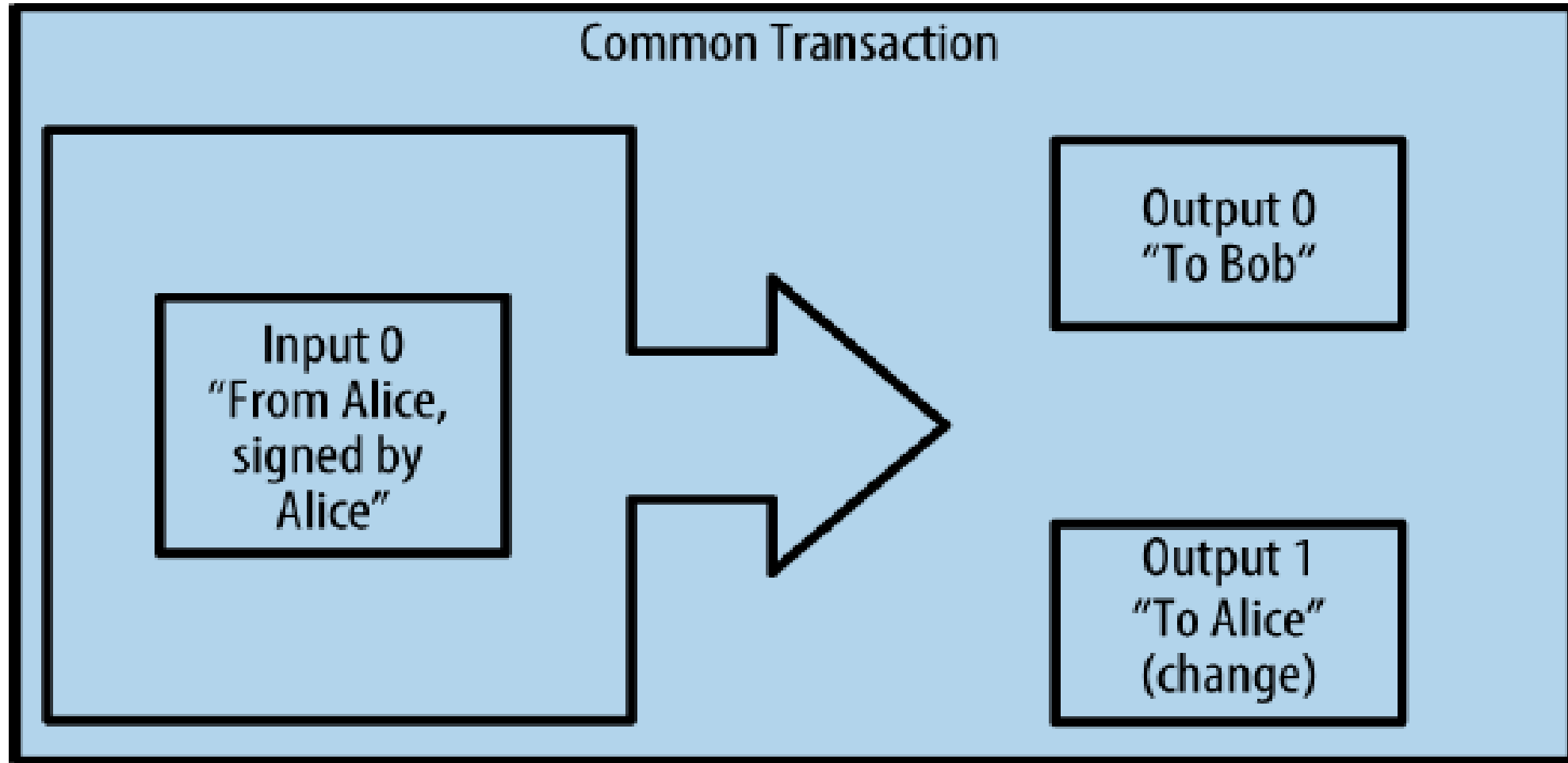
- Transaction contents (input/output)?
- Node (user)/ Public keys?
- Sources/related parties/originals?

Verifying transaction data: Transaction contents

- Verify input-output (looking from database - historical transaction)



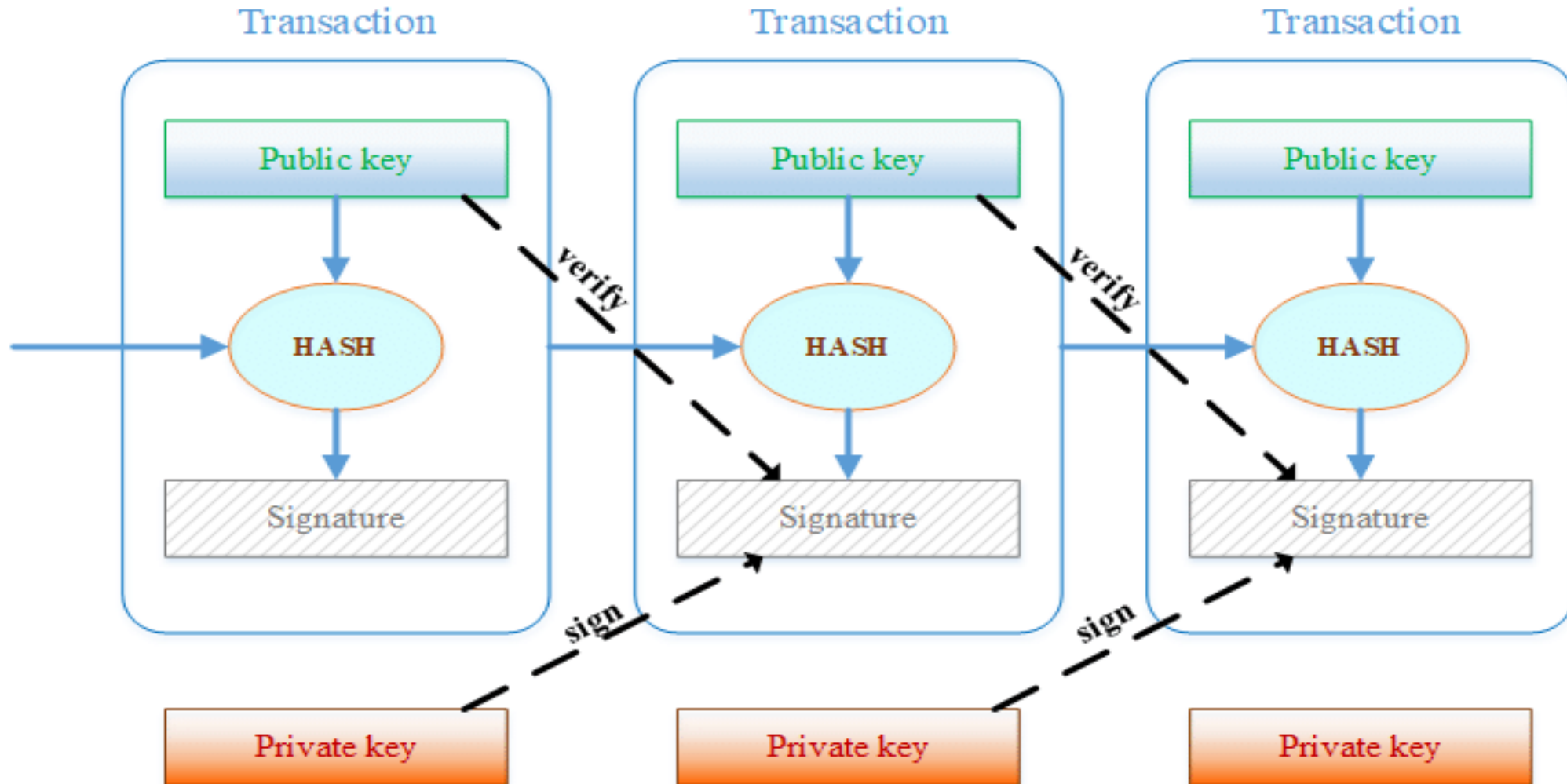
Verifying transaction data: Transaction contents



The structure of transaction in a Bitcoin blockchain

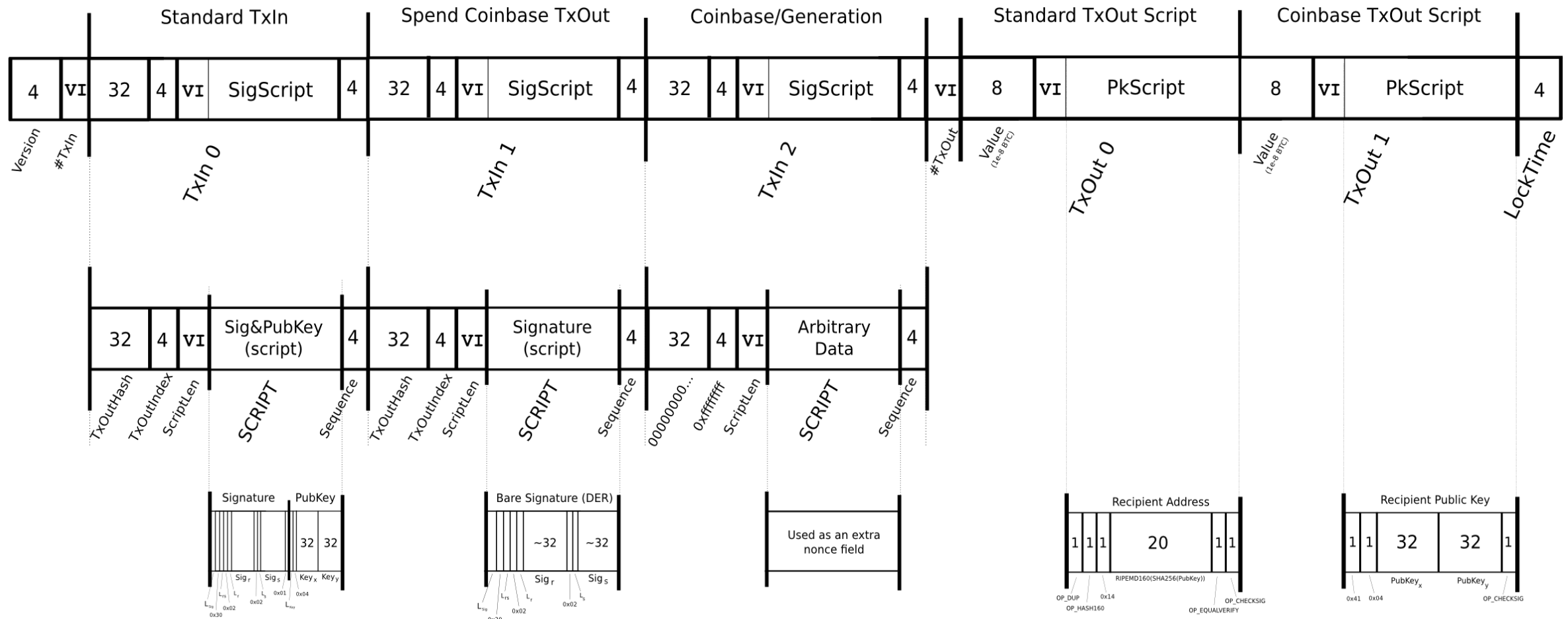
Verifying transaction data: Transaction contents

The structure of transaction in a Bitcoin blockchain



Verifying transaction data: Transaction contents

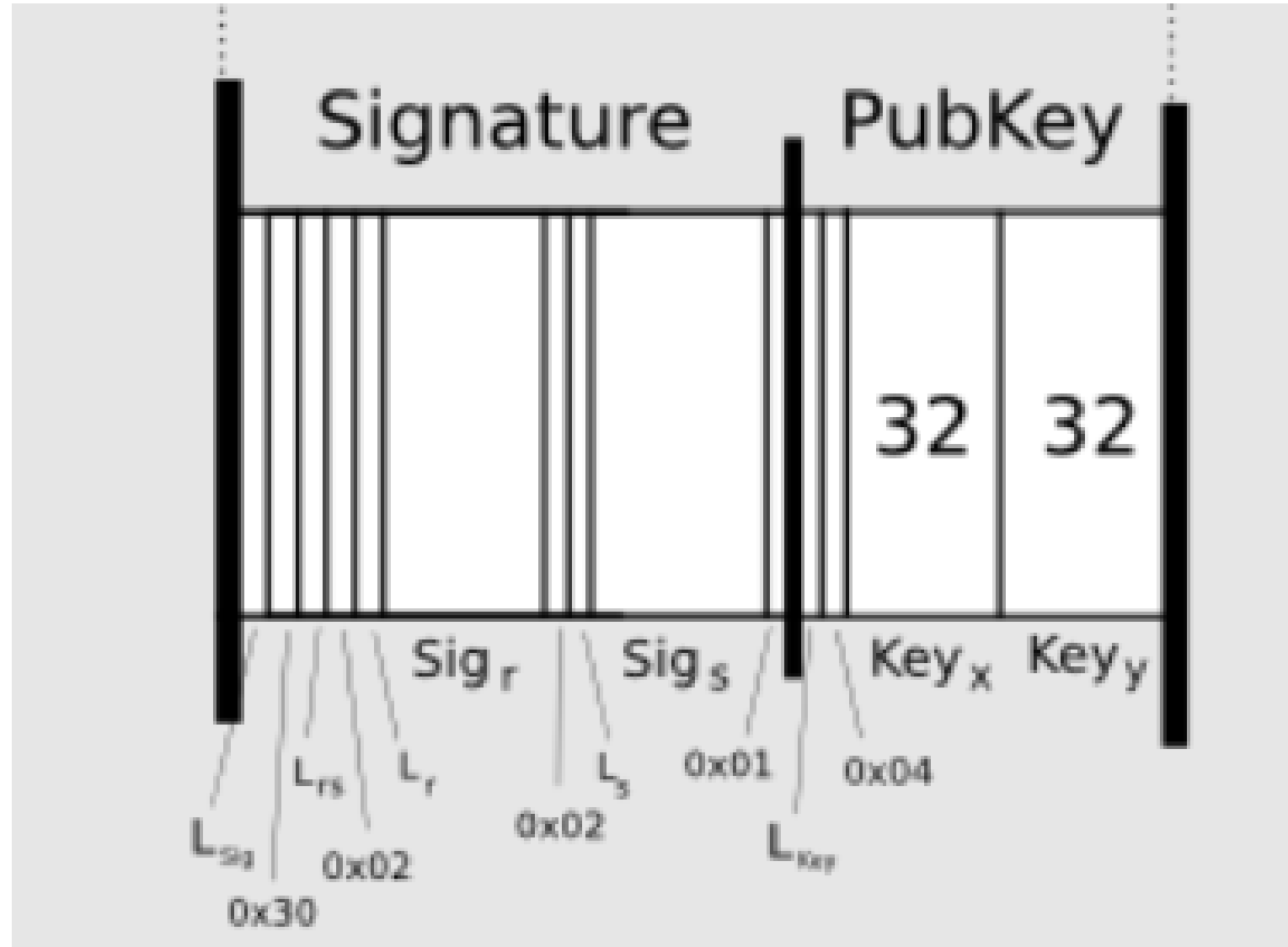
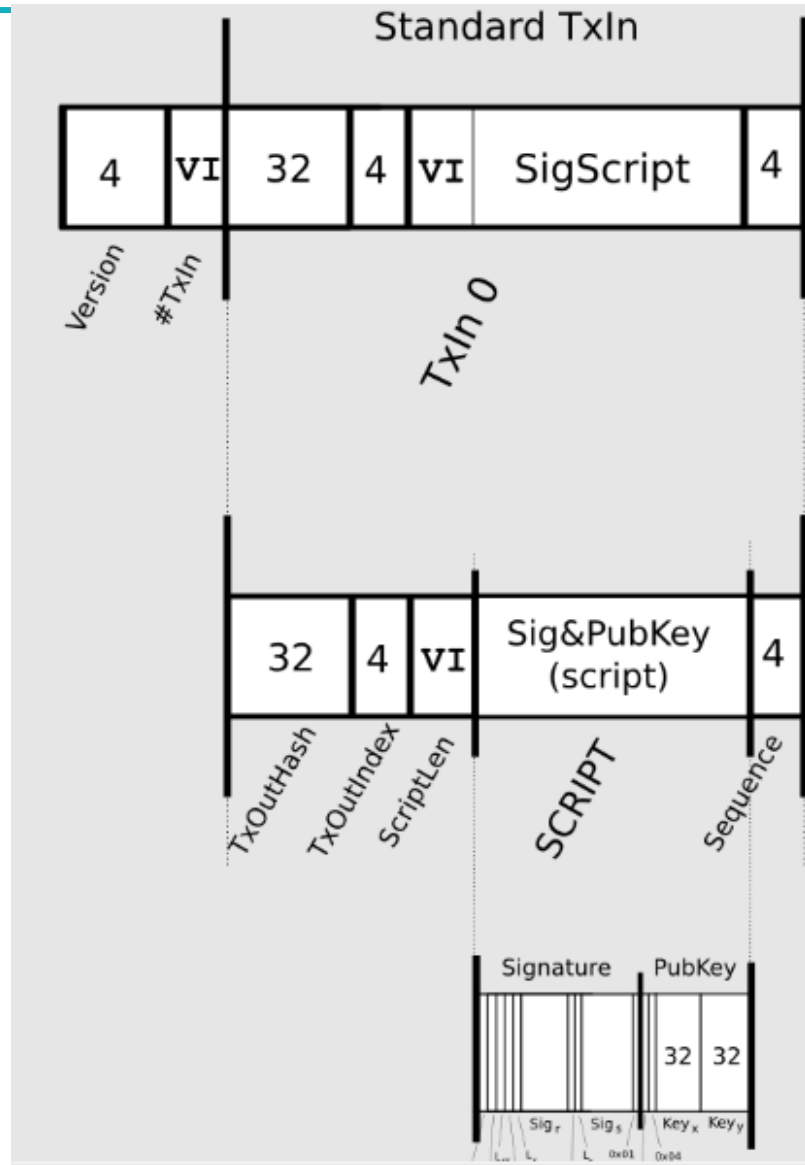
Transaction <https://en.bitcoin.it/w/images/en/e/e1/TxBinaryMap.png>



Scripts and DER encoding both use big-endian values, all other serializations use little-endian

etotheipi@gmail.com / 1Gffm7LKXcNFPrty6yF4JBoe5rVka4sn1

Verifying transaction data: Transaction contents



Elliptic Curve Digital Signature Algorithm (ECDSA)

ECDSA parameters

- Prime number: p (or $f(x)$)
- Curve coefficients: $a, b \in \mathbb{Z}_p$
- Base points: $G \in E(\mathbb{Z}_p)$
- The number $n = \text{ord}(\langle G \rangle)$
- The number $h = \frac{\text{ord}(E(\mathbb{Z}_p))}{n}$
- $H: \{0,1\}^* \rightarrow \{0,1\}^l, l = l(n)$

Key generation (for signer)

- Secret key: $d \in_R [1, n - 1]$
- Public key: $Q = d \cdot G \in E(\mathbb{Z}_p) = (Key_x, Key_y)$ NIST.FIPS.186-5

Key distribution: Curve, Q

Bit length of n	Maximum Cofactor (h)	Comparable Security Strength
224 - 255	2^{14}	approximately $n/2$; at least 112 bits
256 - 383	2^{16}	approximately $n/2$; at least 128 bits
384 - 511	2^{24}	approximately $n/2$; at least 192 bits
≥ 512	2^{32}	approximately $n/2$; at least 256 bits

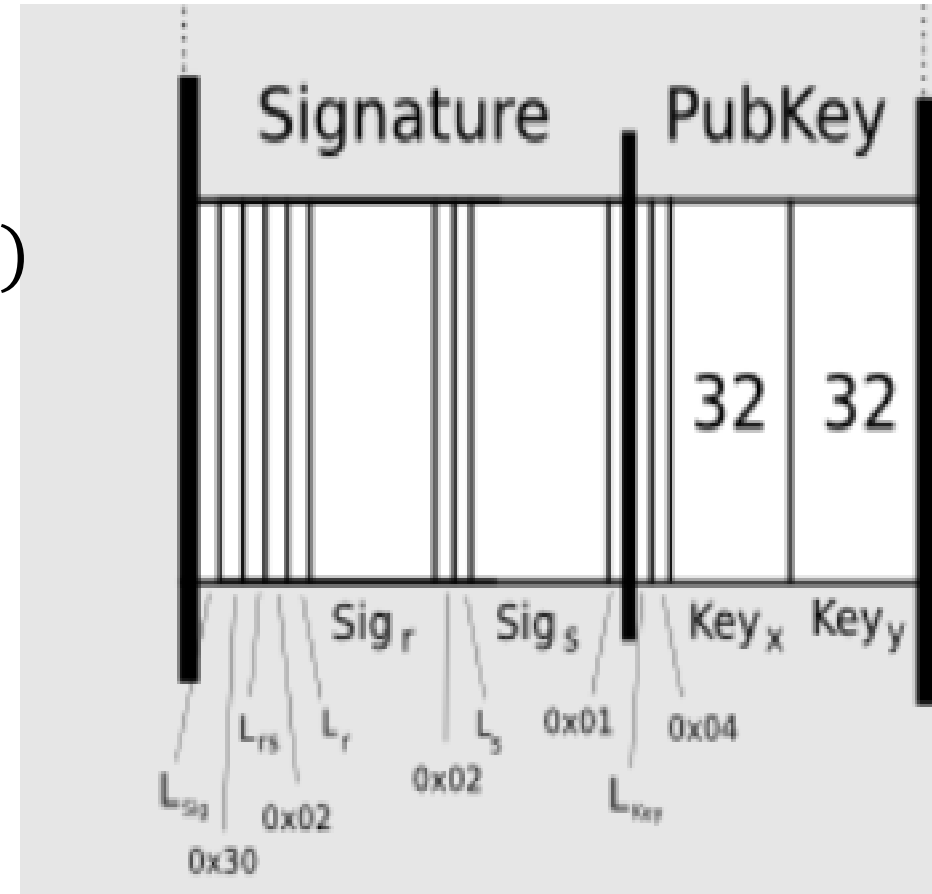
Elliptic Curve Digital Signature Algorithm (ECDSA)

Key generation (for signer)

- Secret key: $d \in_R [1, n - 1]$
- Public key: $Q = d \cdot G \in E(\mathbb{Z}_p) = (Key_x, Key_y)$

Signing (the message m)

- Choose secret for each message:
 $k \in_R [1, n - 1]$
- Compute $R = k \cdot G = (x_1, y_1), r = x_1$
- Compute $s = k^{-1}(H(m) + d \cdot r) \bmod n$
- Output signature (r, s)



L_{sig}	0x03	L_{rs}	0x02	L_r	sig_r	0x02	L_s	sig_s	0x01	L_{key}	0x04	key_x	key_y
-----------	------	----------	------	-------	---------	------	-------	---------	------	-----------	------	---------	---------

Verifying transaction data: Node (user)/ Public keys?

- **Bitcoin wallet address:** Pay-to-Public-Key-Hash (P2PKH)

Open Source JavaScript Client-Side Bitcoin Wallet Generator

Single Wallet

Paper Wallet

Bulk Wallet

Brain Wallet

Vanity Wallet

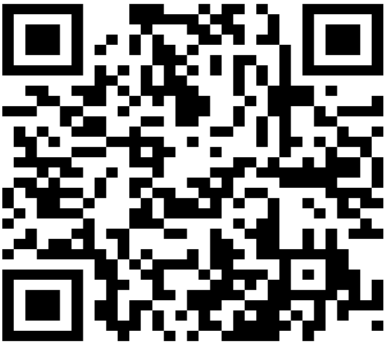
Split Wallet

Wallet Details

Generate New Address

Print

Bitcoin Address



SHARE

195sYZTRik2y3gidQZ3svoU7NexoLPJopr

Private Key



SECRET

L3SRbuMJw4MaX997MN83zRSoYpLofu4Kf3hStWZi77WxnwHvAymm

Verifying transaction data: Node (user)/ Public keys?

- **Bitcoin wallet address:** Pay-to-Public-Key-Hash (P2PKH)

$$A = \text{RIPEMD160}(\text{SHA256}(K))$$

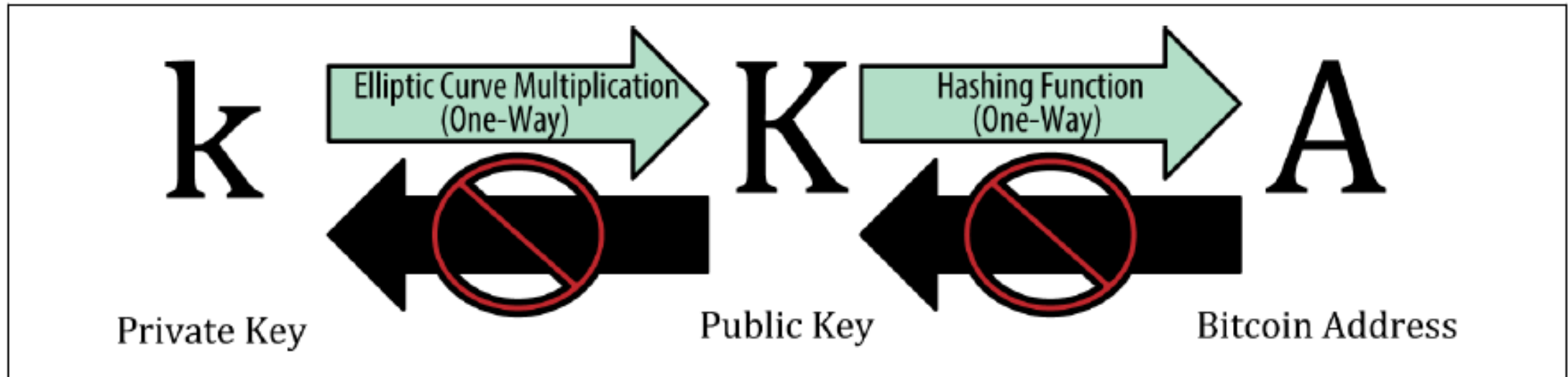


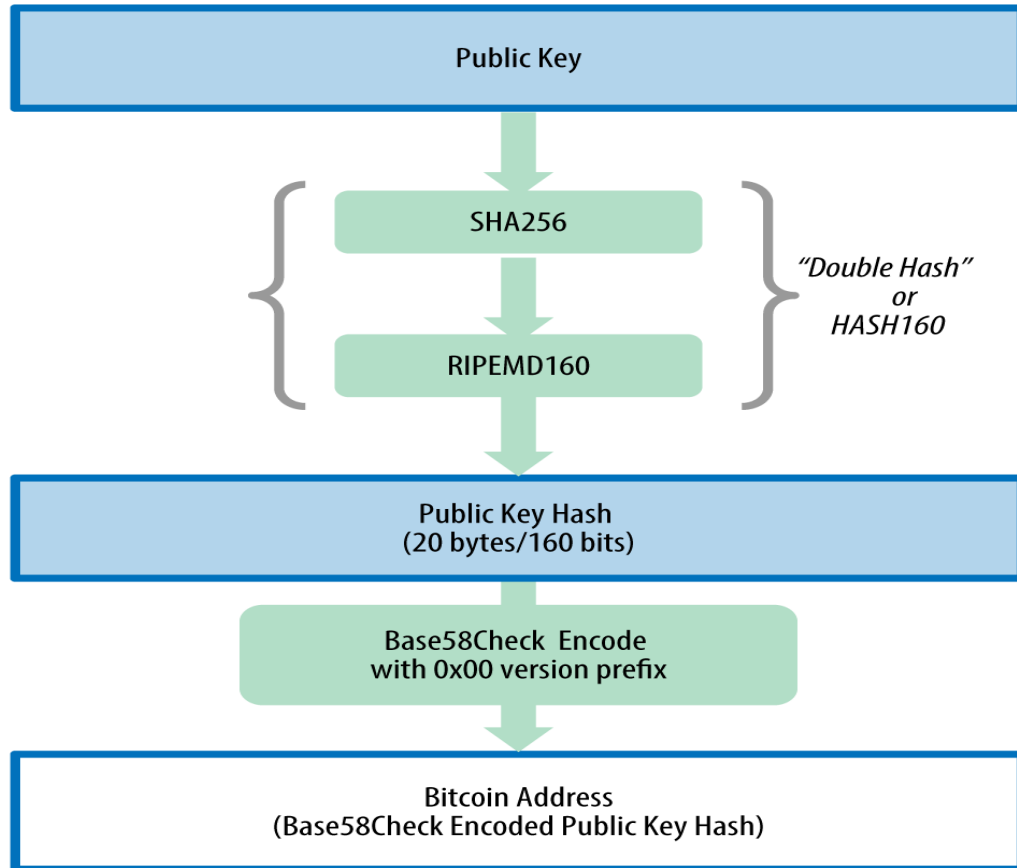
Figure 4-1. Private key, public key, and bitcoin address

hash functions	BLAKE2b , BLAKE2s , Keccak (F1600), SHA-1 , SHA-2 , SHA-3, SHAKE (128/256), SipHash , LSH (128/256), Tiger , RIPEMD (128/160/256/320), SM3 , WHIRLPOOL
----------------	--

Verifying transaction data: Node (user)/ Public keys?

- **Bitcoin wallet address:** Pay-to-Public-Key-Hash (P2PKH)

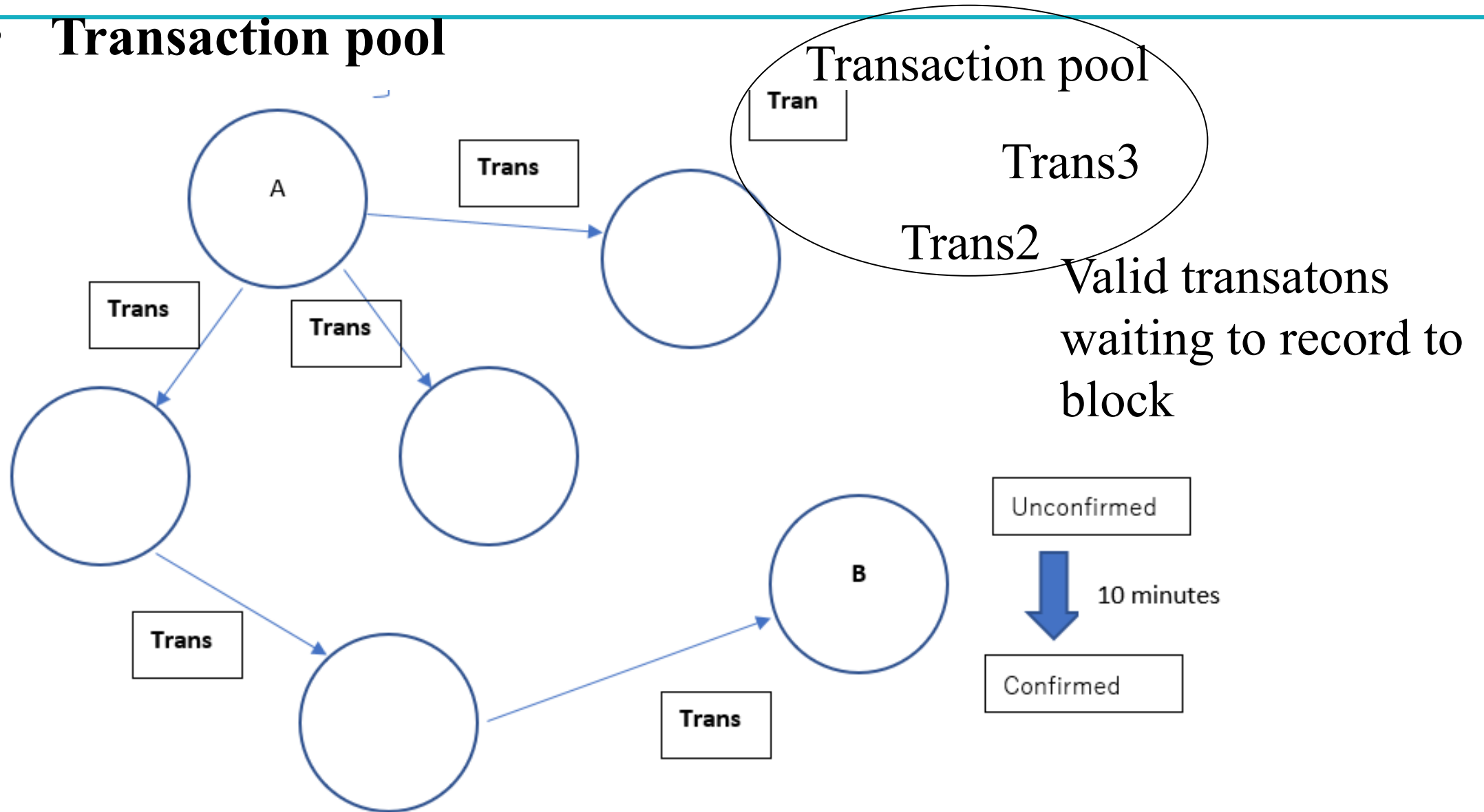
Public Key to Bitcoin Address



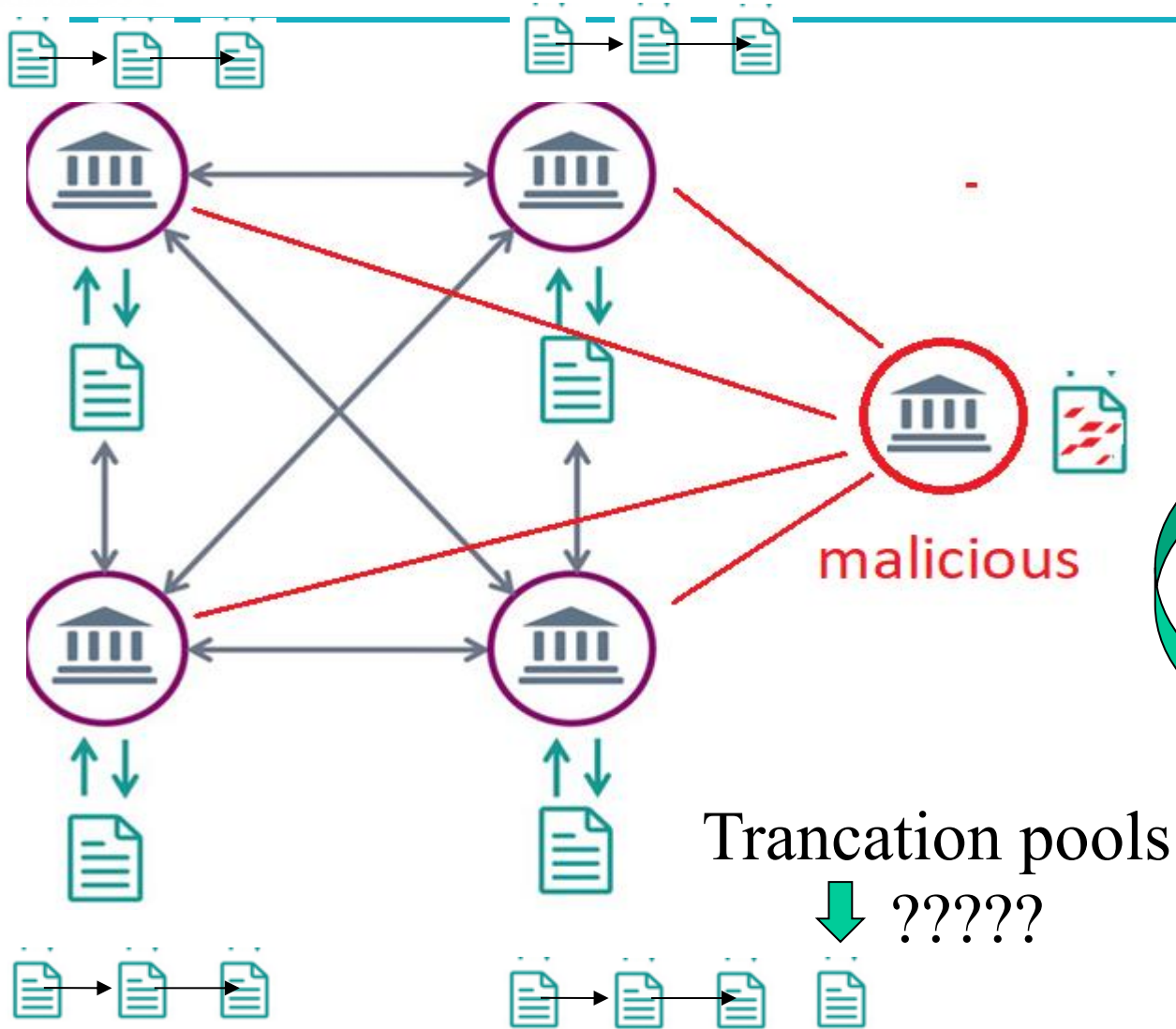
$$A = \text{RIPEMD160}(\text{SHA256}(K))$$

Verifying transaction data: Transaction pools

- Transaction pool



Securely synchronizing



• How to ?

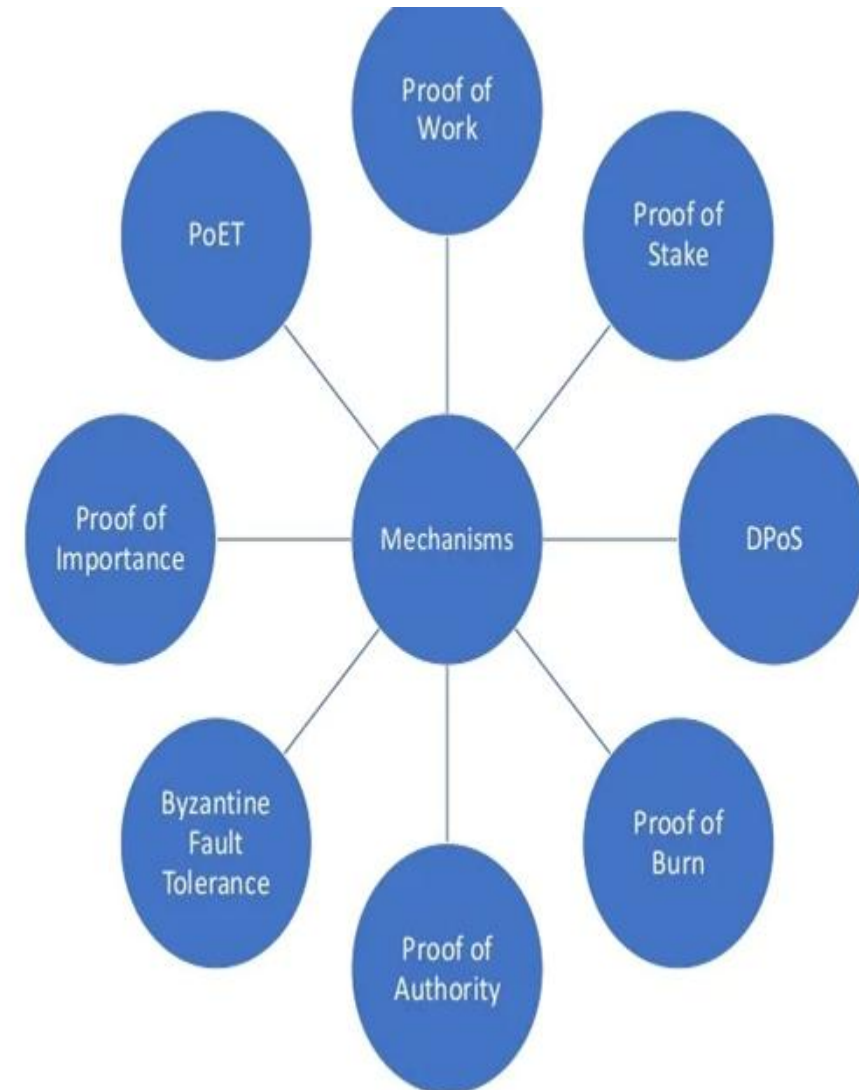
- ✓ Verifying new events/transactions (data authentication, other security features)
- ✓ Securely storage, **create, verify, synchronize a new block** (all nodes): immutability, transability,

Consensus mechanism

Fault-tolerant mechanism

Consensus mechanism:

- Supporting nodes (**create new space to record events (blocks)**, distributing data);
- Securely synchronizing events;



A **consensus mechanism** is a fault-tolerant mechanism to **reach an agreement** on a single state of the network among distributed nodes.

Contents

1. Proof-of-work	212
2. Proof-of-stake	212
3. Delegated proof-of-stake	213
4. Practical Byzantine fault tolerance	216
5. Proof-of-capacity	217
6. Proof-of-elapsed time	218
7. Proof-of-activity	218
8. Proof-of-publication	219
9. Proof-of-retrievability	220
10. Proof-of-importance	221
11. Proof-of-ownership	222
12. Proof-of-burn	222

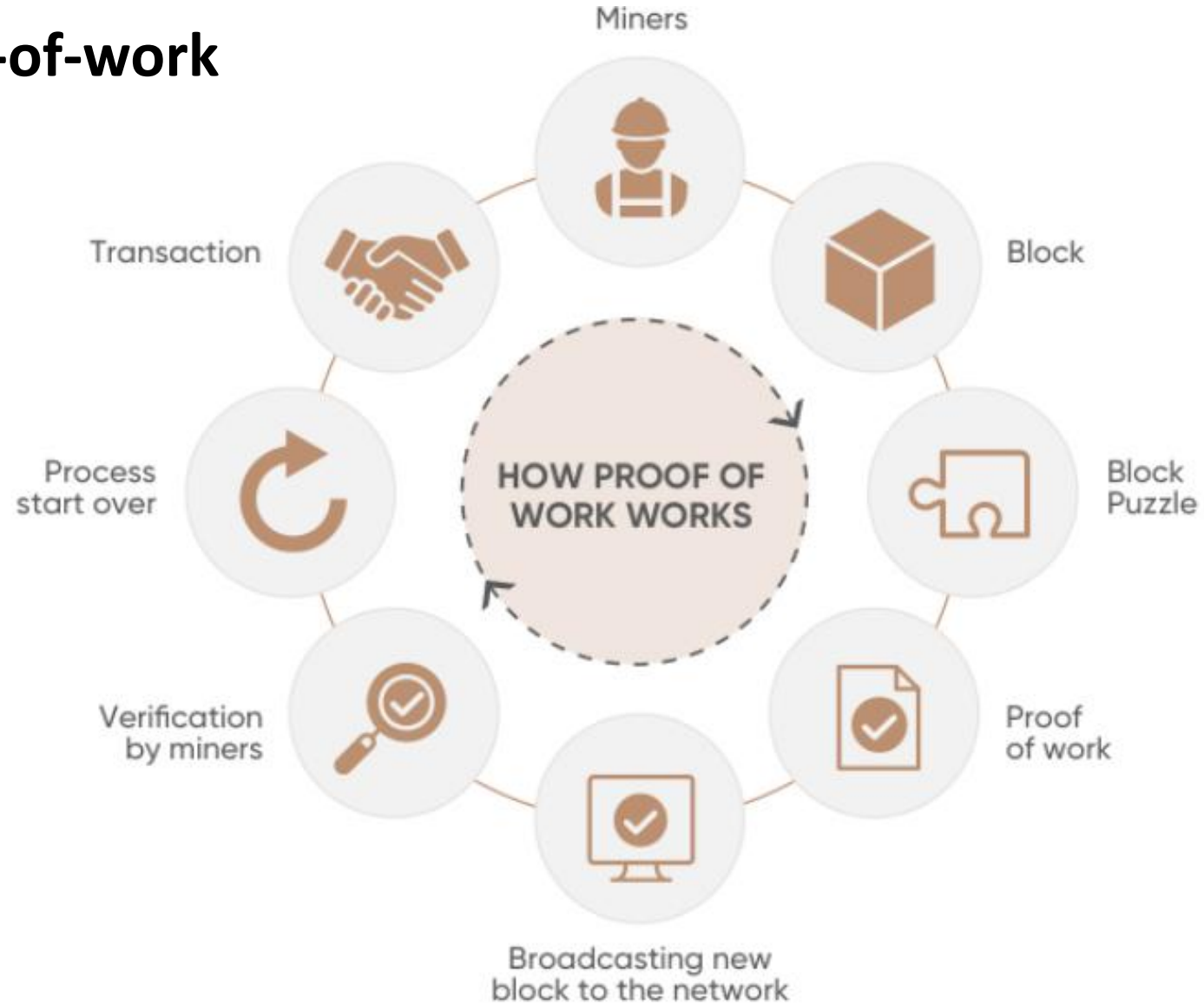
Proof-of-work



- Prevent multiple fake requests
- Trustless and distributed consensus

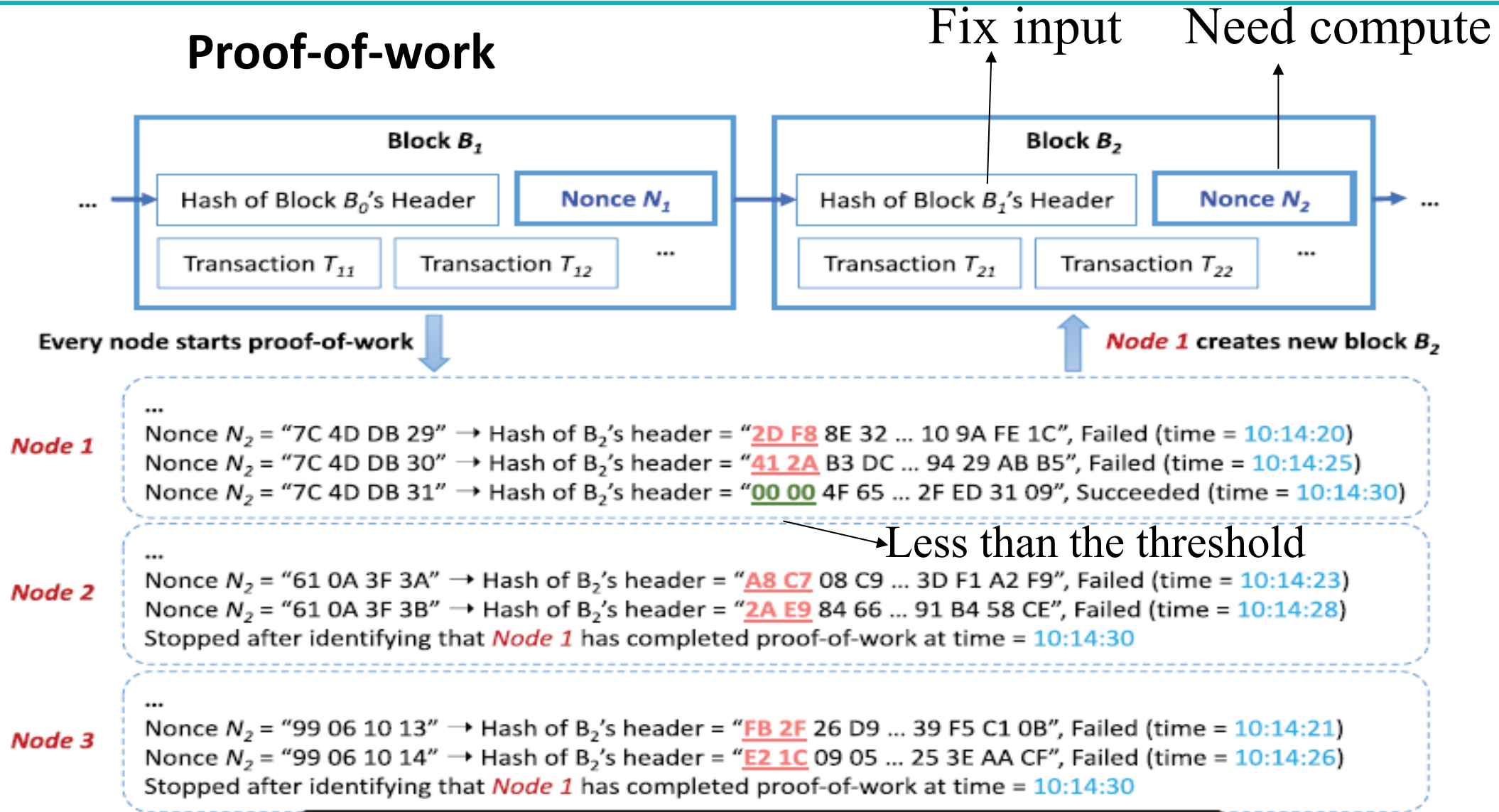
To add each block to the chain, miners must compete to solve a difficult puzzle using their computers processing power.

Proof-of-work



Consensus mechanism

Proof-of-work



Consensus mechanism

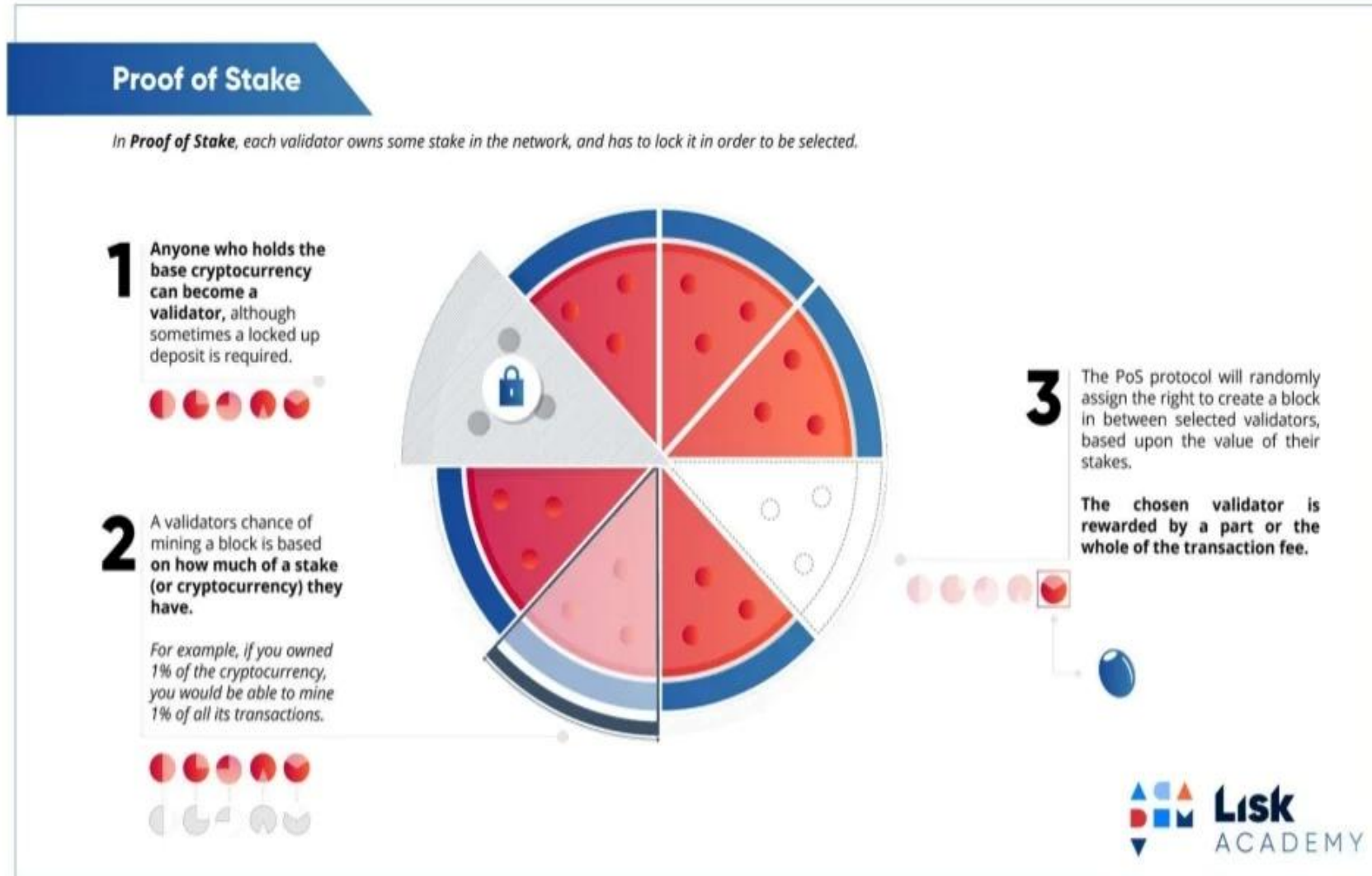
Proof-of-Stake

miners → validators



There is no competition as the block creator is chosen by an algorithm based on the user's stake.

Consensus mechanism

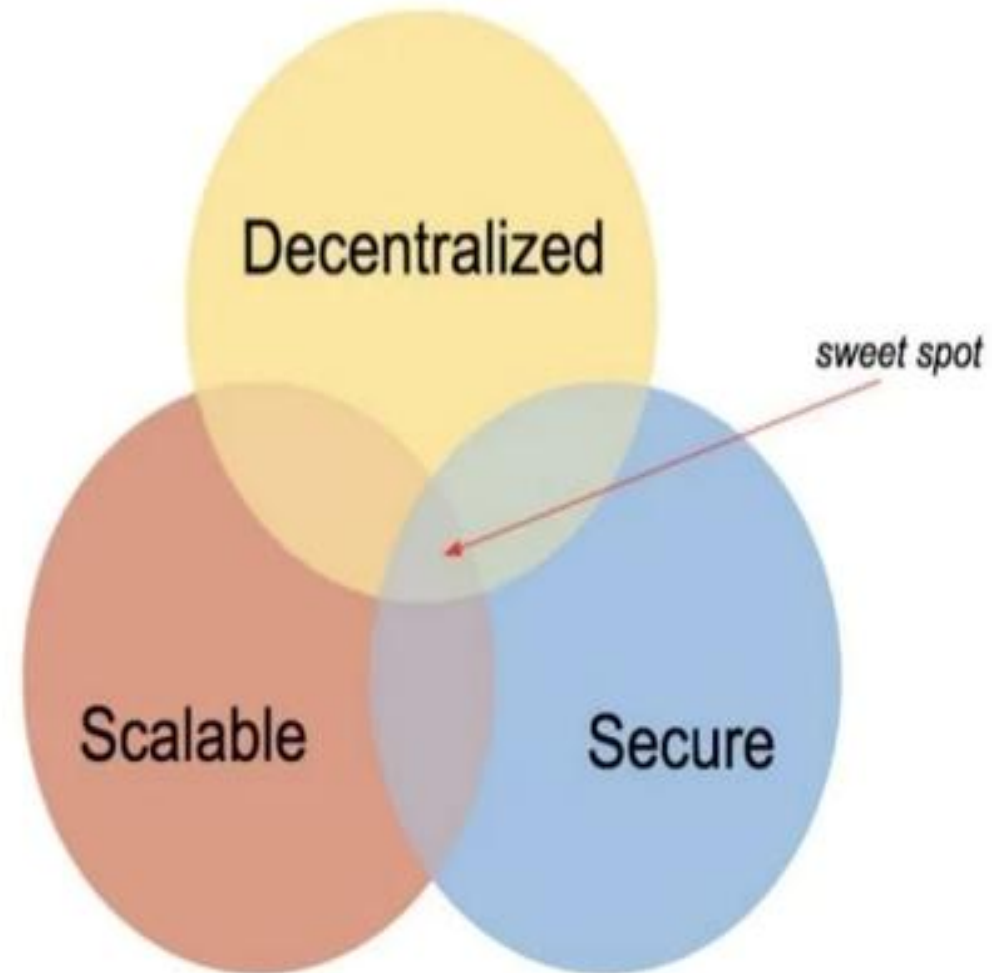


<https://lisk.io/content/5-academy/2-blockchain-basics/4-how-does-blockchain-work/8-proof-of-stake/8-pos-infographic.jpg>

Consensus mechanism

The Scalability Trilemma

- Each of the three goals by itself is "easy" to achieve
- But you have to sacrifice one of Decentralization, Scalability or Security to achieve a high level in the other two
- Bitcoin sacrifices scale (to some degree) for decentralization and security
- Ripple, Stellar, EOS sacrifice decentralization (to some degree) for scale and security
- Related to CAP Theorem for Distributed Data Stores:
 - Consistency
 - Availability
 - Partition Tolerance



Bitcoin: a case study

Decentralization

- + no central point of control
- + consensus protocols
 - participants on the network all must agree unanimously add a data
 - do it while ensuring its integrity;

Immutability (impossible to change)

- + data remains unchangeable once it's been recorded and processed on the blockchain (protected from any modifications or attacks);
- + makes the system more secure (eliminates trust required by traditional centralized authorities)

Bitcoin: a case study

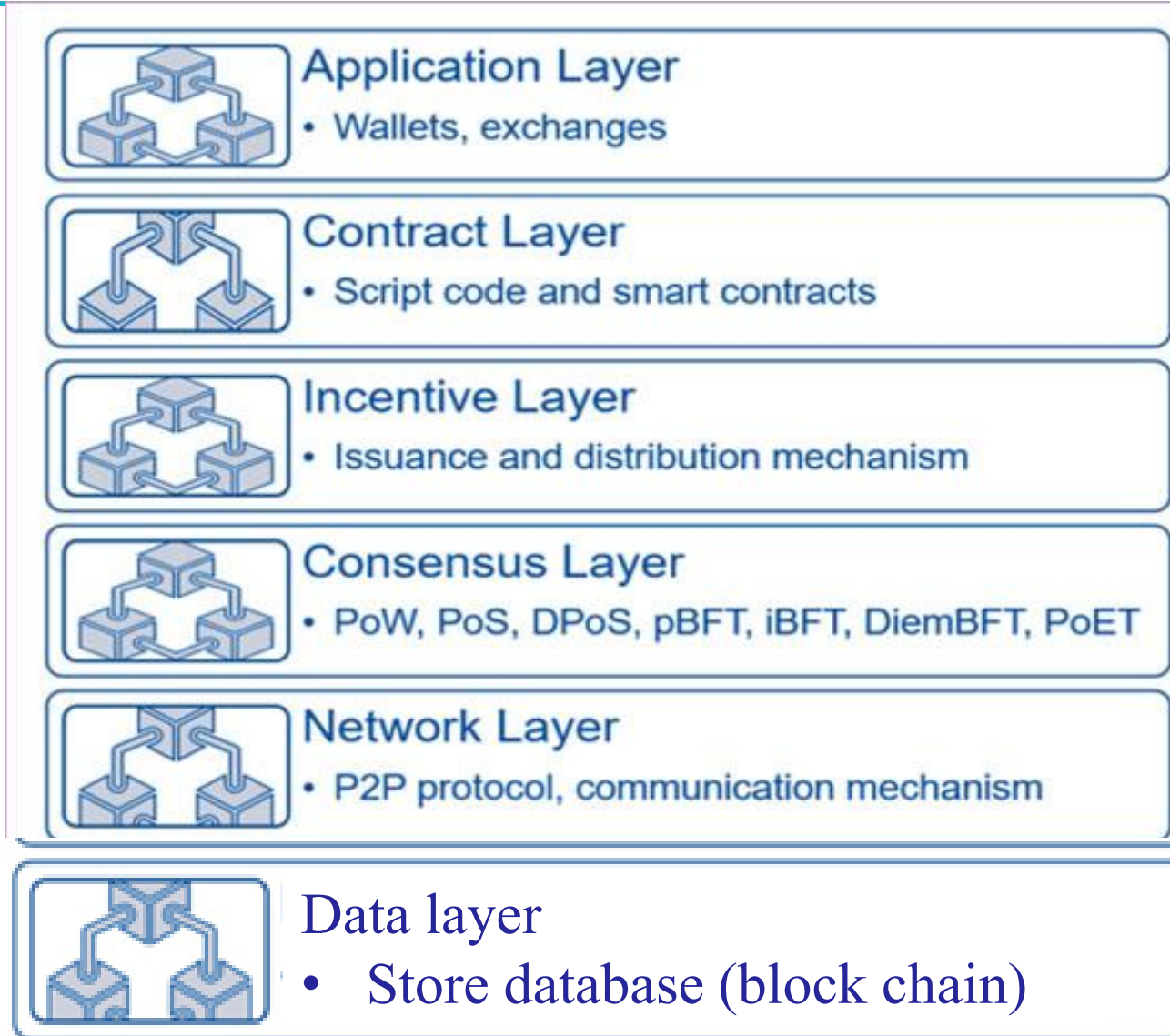
Transparency (data open/transparent)

- + block explorer: access all block datas;
- + search the blocks of a blockchain: access their contents and their relevant details;

Outline

- Network secure protocols
 - Authentication;
 - Key agreement;
 - IPSec; SSL/TLS; SSH; Kerberos
- Blockchain network security
 - Motivation
 - Database structure;
 - Secure Transactions and consensus mechanism;
 - **Network architecture;**
 - Applications

Network Architecture



Programmable
distributed applications

- **How to ?**
 - ✓ Verifying new events/transations (data authentication, other security features)
 - ✓ Securely storage, create, verify, synchronize a new block (all nodes): immutability, transability,

Next generation of blockchain network

- A smart contract is a computer program or a transaction protocol
 - ✓ automatically execute
 - ✓ control or document legally relevant events (**compiled code**)
 - ✓ actions according to the terms of a contract or an agreement (**sending a transaction from a wallet to the blockchain**)

Smart contract

```
1  #![no_std]
2
3  elrond_wasm::imports!();
4
5  #[elrond_wasm::derive::contract]
6  pub trait Adder {
7      #[view(getSum)]
8      #[storage_mapper("sum")]
9      fn sum(&self) -> SingleValueMapper<BigInt>;
10
11     #[init]
12     fn init(&self, initial_value: BigInt) {
13         self.sum().set(&initial_value);
14     }
15
16     #[endpoint]
17     fn add(&self, value: BigInt) -> SCResult<()> {
18         self.sum().update(|sum| *sum += value);
19
20         Ok(())
21     }
22 }
```

<https://docs.near.org/docs/develop/contracts/rust/intro>

